

Architectures Parallèles

TP1 ANNEX: AARCH64 CHEATSHEET

UVSQ - M1 CHPS

Some of you may have an ARM processor on your machine (most likely Apple Silicon). This annex document is an AArch64 version of the introductory cheatsheet.

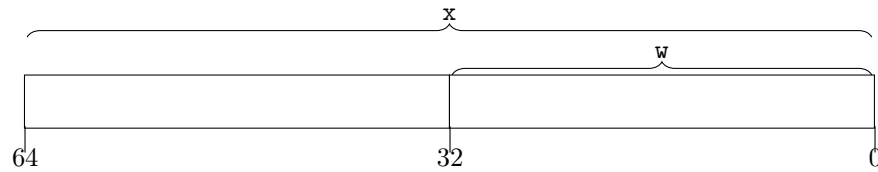
0 Cheatsheet

0.1 Registers

The following table lists the "general purpose" registers (GPRs) of the AArch64 architecture:

64 bits	Comment	Volatile	32 bits
x0	Functions arguments and return	✓	w0
x1	Functions arguments and return	✓	w1
x2	Functions arguments and return	✓	w2
x3	Functions arguments and return	✓	w3
x4	Functions arguments and return	✓	w4
x5	Functions arguments and return	✓	w5
x6	Functions arguments and return	✓	w6
x7	Functions arguments and return	✓	w7
x8	Functions return for > 128 bits types, otherwise scratch	~	w8
x9		✓	w9
x10		✓	w10
x11		✓	w11
x12		✓	w12
x13		✓	w13
x14		✓	w14
x15		✓	w15
x16		✓	w16
x17		✓	w17
x18		✓	w18
x19			w19
x20			w20
x21			w21
x22			w22
x23			w23
x24			w24
x25			w25
x26			w26
x27			w27
x28			w28
x29	Frame pointer		w29
x30	Functions return address (do not use)		w30

Depending on the size of the manipulated types, these registers can be accessed in 64 or 32 bits modes. The 32 bits variant corresponds to the **lower part** of the register:



0.2 Syntax

Similarly to x86's Intel syntax, AArch 64 uses the "destination $\leftarrow$ sources" scheme.

Example: "mov rax, #42" stores the value 42 in the rax register. Note how this instruction would not make sense in Intel AT&T syntax.

Moreover, immediate operands are preceded by a #, and memory operands are put in brackets [].

0.3 Instructions

The following table lists some basic instructions:

Instruction	Description	C equivalent
add x0, x1, x2	Integer addition	x0 = x1 + x2;
sub x0, x1, x2	Integer subtraction	x0 = x1 - x2;
mul x0, x1, x2	Integer multiplication	x0 = x1 * x2;
mov x0, x1	Copy content of a register into another	x0 = x1;
ldr x0, [x1]	Load value from memory to a register	x0 = *x1;
str [x0], x1	Store value from a register into memory	*x0 = x1;
cmp x0, x1	Compare operands and set the flag register	x0 == x1
b.eq addr	Read the flag register and jump if equal	
b.ne	Read the flag register and jump if non-equal	
b addr	Jump unconditionally	

A more complete list of AArch64 instructions can be found at <https://developer.arm.com/documentation/ddi0602/2025-09/Base-Instructions>.

Instruction operands can be of the following types:

Operand type	Examples	C equivalent
Immediate	mov x0, #42 mul x0, x0, #2	x0 = #42; x0 *= 2;
Register	mov x0, x1 add x0, x1, x1	x0 = x1; x0 += x1 + x1;
Memory	str [x0], x1 mov x0, [x1, #16]	*x0 = x1; x0 = *(x1+16);

Finally, the **address** operand of the break instructions corresponds to the address of the instruction to jump to. In practice, it is easier to define *labels* in the following way:

```
b.eq mylabel
[...]
mylabel:
mov ..., ...
```

If the flag register is set to equal, execution will jump to the mov instruction below mylabel.